LESSON 02 · CHEAT SHEET

Functional Interfaces: Rules and Standards

java.util.function · Java 8 → Java 21
examples use the shared course dataset

The SAM rule

1. **Exactly one abstract method** = functional interface = can be a lambda or method reference target.
2. **Do not count:** `default` 8, `static` 8, `private` 9 methods, and public `Object` methods (`equals`, `hashCode`, `toString`).
3. **Generic SAMs are fine:** `Converter<A,B> { B convert(A a); }`
4. **@FunctionalInterface** is optional: it adds a compile-time check and documents intent. Always use it on your own.
5. **Older SAMs work too:** Runnable 1.0, Comparator 1.2, FileFilter 1.2, Callable 5.

Canonical snippets

1 · A domain functional interface

```
@FunctionalInterface
public interface FeeRule {
    BigDecimal feeFor(Transaction txn); // SAM
    default FeeRule plus(FeeRule o) { ... }  // [8]
    static FeeRule flat(String a) { ... }   // [8]
    private static BigDecimal round(...) {} // [9]
    boolean equals(Object o);  // not counted
}
// T003 1221.40 -> 0.50 + 0.1% = 1.72
```

2 · Compose instead of writing classes Java 8

```
Predicate<Payment> SETTLE_TODAY = SUCCESSFUL
    .and(AT_LEAST_100)
    .and(PAYPAL.negate());   // [P001, P005, P010]

ADD_FEE.andThen(ADD_GST).apply(hundred);  // 110.09
ADD_FEE.compose(ADD_GST).apply(hundred);  // 110.00
steps.stream()
    .reduce(Function.identity(), Function::andThen);
```

3 · Primitive, no boxing

```
ToIntFunction<Flight> BOOKED = Flight::booked;
flights.stream().mapToInt(BOOKED).sum(); // 1808
```

Custom or built-in?

Write your own when

- the name adds domain meaning
- it must throw a checked exception
- 3+ parameters (TriFunction)

Reuse built-in when

- the shape is generic
- it crosses APIs (streams, Optional)
- it is local or numeric

Catalogue (all Java 8)

Interface	Method	Shape	Use
`Supplier<T>`	get	`() → T`	lazy value, factory
`Consumer<T>`	accept	`T → void`	print, send, log
`BiConsumer<T,U>`	accept	`(T,U) → void`	Map.forEach
`Function<T,R>`	apply	`T → R`	map, convert
`BiFunction<T,U,R>`	apply	`(T,U) → R`	two inputs
`UnaryOperator<T>`	apply	`T → T`	same-type change
`BinaryOperator<T>`	apply	`(T,T) → T`	reduce, merge
`Predicate<T>`	test	`T → boolean`	filter
`BiPredicate<T,U>`	test	`(T,U) → boolean`	check a pair
`IntPredicate`	test	`int → boolean`	no boxing
`ToIntFunction<T>`	applyAsInt	`T → int`	mapToInt
`IntUnaryOperator`	applyAsInt	`int → int`	int math
`ObjIntConsumer<T>`	accept	`(T,int) → void`	print a pair

Composition methods

Method	Meaning	Since
`f.andThen(g)`	f first, then g	8
`f.compose(g)`	g first, then f	8
`Function.identity()`	returns its input	8
`p.and / or / negate`	logic on predicates	8
`Predicate.not(p)`	static negate, works on method refs	11
`Predicate.isEqual(x)`	Objects.equals(x, t)	8
`Consumer.andThen`	run both, in order	8
`BinaryOperator.minBy/maxBy`	from a Comparator	8

Do / Don't

✔ Do

- Annotate your SAMs
- Role noun + verb: `FeeRule.feeFor`
- Primitive types for numbers
- Keep Functions pure

✘ Don't

- Overload `run(Supplier)` + `run(Callable)`: ambiguous
- Add a 2nd abstract method
- `Function<T,Integer>` in hot paths

Lesson 02 · sticky notes

Cut out, stick on your wall, and read one every morning.

1 abstract method

That is the whole rule.
default, static, private and Object methods don't count.

@FunctionalInterface

Optional, but always use it.
Compiler catches a 2nd abstract method.

4 shapes

Supplier () → T
Consumer T → void
Function T → R · Predicate T → boolean

andThen vs compose

f.andThen(g): f first.
f.compose(g): g first.
`110.09` vs `110.00`

No boxing!

Numbers? Use IntPredicate, ToIntFunction, IntUnaryOperator.
`mapToInt(Flight::booked)`

Supplier vs Callable

Same shape () → T.
Only Callable.call() may throw a checked exception.

Custom SAM when...

domain name · checked exception · 3+ params (TriFunction)

Never overload by SAM

`run(Supplier)` + `run(Callable)`
= "reference to run is ambiguous"